

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Technical Presentation

Code of Conduct:

I AGNEESHWARAN B / 192524508 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: AGNEESHWARAN B

Technical Presentation

- 1. Reactive Agent-Based Traffic Control System**
Prepare a technical presentation on designing a **Reactive Agent** for a smart traffic control system. Explain how the agent responds to real-time traffic inputs, manages signal changes, and ensures minimal congestion using streaming data processing.
- 2. Goal-Based Agent for Healthcare Diagnosis**
Develop a presentation on a **Goal-Based Agent** used in an AI healthcare system. Describe how the agent sets diagnostic goals, evaluates patient data, and selects optimal treatment pathways using decision-making logic and data pipelines.
- 3. Utility-Based Agent for E-Commerce Recommendations**
Create a technical presentation on a **Utility-Based Agent** that generates personalized product recommendations. Explain how the agent assigns utility values to different options and optimizes user satisfaction using large-scale data processing.
- 4. Learning Agent for Personalized Education System**
Present the design of a **Learning Agent** in an AI-driven educational platform. Explain how the agent improves performance over time using feedback, adapts content difficulty, and integrates with distributed data processing systems like PySpark.
- 5. Multi-Agent System for Disaster Management**
Prepare a presentation on a **Multi-Agent System** where multiple agents (sensor agents, decision agents, communication agents) collaborate for disaster detection and response. Explain coordination strategies, data sharing, and real-time decision-making.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

ANSWER SHEET – TECHNICAL PRESENTATION

C05: AI-Based Systems using Intelligent Agents, PySpark and Distributed Processing

1. Reactive Agent-Based Traffic Control System

Introduction:

A Reactive Agent-Based Traffic Control System is an intelligent system in which a traffic-control agent observes real-time traffic conditions and immediately changes traffic signals according to the current situation. The main objective is to reduce congestion, waiting time and unnecessary fuel consumption.

Architecture:

Traffic Sensors/Cameras → Streaming Data Collection → PySpark/Streaming Processing → Reactive Agent → Signal Controller → Traffic Signals → New Traffic Data.

Working:

1. Sensors and cameras continuously collect vehicle count, queue length, vehicle speed and lane occupancy.
2. Streaming data is processed in real time using a distributed processing framework such as PySpark.
3. The reactive agent compares the current traffic condition with predefined rules.
4. If one road has heavy traffic, the agent increases its green-signal duration.
5. If traffic is low, the green duration can be reduced and allocated to another direction.
6. The updated signal decision is sent to the traffic signal controller.
7. The process repeats continuously as new traffic data arrives.

Example:

If the north-south road has 80 vehicles waiting while the east-west road has only 20 vehicles, the agent can provide a longer green time to the north-south direction. When the traffic condition changes, the signal timing is changed again.

Benefits:

- Real-time response to traffic conditions.
- Reduced congestion and waiting time.
- Better utilization of road capacity.
- Scalable processing for data from many sensors.
- Automatic decision-making without continuous human intervention.

Conclusion:

The reactive agent provides fast rule-based responses to changing traffic conditions. Combining intelligent agents with streaming and distributed processing makes the traffic-control system scalable and suitable for smart-city applications.

2. Goal-Based Agent for Healthcare Diagnosis

Introduction:

A Goal-Based Agent in healthcare makes decisions by working toward a specific goal, such as identifying a possible diagnosis and selecting an appropriate treatment pathway. It evaluates patient information and chooses actions that help achieve the diagnostic goal.

Architecture:

Patient Data → Data Pipeline → Data Processing → Goal-Based Agent → Diagnostic Evaluation → Treatment Options → Recommended Pathway.

Working:

1. Patient information such as symptoms, age, medical history and laboratory results is collected.
2. A data pipeline prepares and organizes the information.
3. The agent defines a goal, such as identifying the most likely condition.
4. It evaluates the available patient data against possible diagnostic conditions.
5. Candidate diagnoses and treatment pathways are compared using decision-making logic.
6. The agent selects the pathway that best satisfies the defined goal.
7. The result can be presented to healthcare professionals for review.

Example:

For a patient reporting fever, cough and breathing difficulty, the system processes the symptoms and available test results. It evaluates possible conditions and identifies the diagnostic pathway that best matches the available evidence.

Benefits:

- Goal-oriented decision-making.
- Faster processing of large patient datasets.
- Consistent evaluation of available information.
- Supports healthcare professionals in decision-making.
- Can be integrated with distributed data-processing systems.

Important Note:

The agent should be used as a decision-support system. Final diagnosis and treatment decisions must remain under qualified medical supervision.

Conclusion:

A goal-based healthcare agent uses patient data and decision logic to work toward diagnostic objectives. Distributed data pipelines can help it process large amounts of healthcare information efficiently.

3. Utility-Based Agent for E-Commerce Recommendations

Introduction:

A Utility-Based Agent recommends products by assigning a utility value to available choices. The utility value represents how suitable a product is for a particular user. The agent attempts to maximize user satisfaction rather than simply selecting any product.

Architecture:

User Data → Distributed Data Processing → Feature Extraction → Utility-Based Agent → Utility Calculation → Ranking → Personalized Recommendations.

Working:

1. The system collects user information such as search history, purchases, ratings and product interactions.
2. Large datasets are processed using scalable technologies such as PySpark.
3. Features such as price preference, category interest and previous ratings are extracted.
4. The agent calculates a utility score for candidate products.
5. Products are ranked according to their utility scores.
6. The highest-ranked products are displayed as personalized recommendations.
7. User feedback is collected and can be used to improve future recommendations.

Example:

If a user frequently purchases low-cost sports shoes and gives high ratings to comfortable products, the agent can assign higher utility values to affordable and highly rated sports shoes.

Benefits:

- Personalized recommendations.
- Efficient processing of large-scale customer data.
- Better user satisfaction.
- Improved product discovery.
- Supports scalable recommendation systems.

Conclusion:

A utility-based agent makes recommendations by evaluating and ranking alternatives according to their expected usefulness to the user. Large-scale distributed processing helps the system handle millions of products and user interactions.

4. Learning Agent for Personalized Education System

Introduction:

A Learning Agent in an educational platform improves its behavior using feedback and experience. It observes student performance, learns from previous interactions and adapts the difficulty and type of learning content.

Architecture:

Student Interaction → Data Collection → PySpark Distributed Processing → Learning Agent → Performance Analysis → Content Adaptation → Student Feedback → Learning Update.

Working:

1. The system collects marks, quiz results, time spent, completed lessons and student feedback.
2. PySpark can process learning data from many students in a distributed environment.
3. The learning agent analyzes the student's current performance.
4. It identifies strengths and weak areas.

5. The agent adjusts content difficulty according to the student's learning level.
6. Easier material can be provided when the student struggles, while advanced content can be provided after successful performance.
7. New feedback is continuously used to improve future recommendations.

Example:

If a student repeatedly answers basic mathematics questions correctly but struggles with advanced problems, the agent can provide intermediate practice questions and explanations before moving to advanced questions.

Benefits:

- Personalized learning experience.
- Continuous adaptation based on feedback.
- Supports large numbers of students.
- Improves identification of weak topics.
- Reduces the need for identical content for every learner.

Conclusion:

A learning agent creates an adaptive educational environment by learning from student feedback and performance. PySpark enables scalable processing of educational data, making the approach suitable for large online learning platforms.

5. Multi-Agent System for Disaster Management

Introduction:

A Multi-Agent System (MAS) for disaster management consists of multiple intelligent agents that cooperate to detect disasters, share information and coordinate emergency responses. Different agents can perform specialized tasks.

Architecture:

Sensors/Cameras → Sensor Agents → Communication/Data Layer → Decision Agents → Coordination → Response Agents → Emergency Services.

Types of Agents:

- Sensor Agents: collect information such as temperature, smoke, water level, seismic activity or structural conditions.
- Decision Agents: analyze incoming information and determine the severity and appropriate response.
- Communication Agents: share alerts and decisions between agents, control centers and response teams.
- Response Agents: support actions such as evacuation alerts, resource allocation and emergency coordination.

Working:

1. Sensor agents continuously collect real-time environmental information.
2. The collected data is shared through a communication layer.
3. Decision agents analyze the information and identify possible disaster conditions.
4. If a serious event is detected, agents exchange information to confirm the situation.
5. The system determines priorities such as affected areas, evacuation requirements and available resources.
6. Communication agents distribute alerts to relevant teams.
7. Response agents coordinate emergency actions.
8. New sensor information is continuously used to update decisions.

Coordination Strategies:

- Information sharing between agents.
- Task allocation according to agent capabilities.
- Priority-based emergency response.
- Continuous communication and status updates.
- Distributed decision-making to avoid dependence on one component.

Benefits:

- Faster disaster detection.
- Real-time coordination.
- Parallel handling of multiple tasks.
- Better resource utilization.
- Scalable architecture for large disaster areas.

Conclusion:

A multi-agent disaster-management system combines specialized agents to detect, analyze and respond to

emergencies. Cooperation and real-time data sharing allow the system to make faster and more coordinated decisions.